Cálculo de PI usando threads - Relatório

Aluna: Suelle Ferreira Maciel

Implementação

O programa `calcpi.c` calcula π somando 10^9 termos da série de Leibniz-Gregory, divididos igualmente entre N threads POSIX, sem nenhum mecanismo de sincronização.

$$\frac{\pi}{4} = \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1}$$

- **Divisão do trabalho:** cada thread `i` recebe um bloco contíguo de 10^9 / N termos, de `i × bloco` até `(i + 1) × bloco`; a última thread fica com o resto da divisão.
- **Sem sincronização:** cada thread escreve apenas na posição `result[i]` do vetor de resultados parciais. Como nenhuma posição é compartilhada, não há condição de corrida. A thread principal espera todas com `pthread_join` e só então soma o vetor e imprime π.
- **Variáveis locais:** dentro da thread, o número de termos, o número de threads, a soma parcial e o sinal $(-1)^n$ ficam em variáveis locais, na pilha da thread. O valor global `result[i]` é escrito uma única vez, no final.
- **Precisão:** todos os cálculos usam `double`. O resultado obtido foi 3,141592652589..., com erro absoluto de cerca de 10^{-9} em relação a `M_PI`, o esperado para 10^9 termos.
- **Compilação:** `gcc -O2 calcpi.c -o calcpi -lm -lpthread`; execução: `time ./calcpi <num_threads>`.

Plataformas e metodologia

Os testes rodaram em duas máquinas Linux com configurações diferentes: uma com um único núcleo físico e outra com dois.

Plataforma	Ambiente	Processador	CPUs lógicas	Núcleos físicos	Threads por núcleo
1 — mono-processada	GitHub Codespaces (VM em nuvem)	AMD EPYC 7763	2	1	2 (hyperthreading)
2 — dual-processada	VM Linux em nuvem	Intel Xeon @ 2,10 GHz	2	2	1

- Cada configuração (1, 2, 4, 8, 16 e 32 threads) foi executada 5 vezes, medindo o tempo real (*wall clock*) com o comando `time` do bash.
- Antes das medições, a carga da máquina foi verificada (carga média 0,00 na plataforma 2).
- Para cada configuração foram calculados a média, o desvio-padrão amostral e o coeficiente de variação (CV = desvio / média). Na plataforma 2, a série com 4 threads deu CV de 5,4% e foi refeita, ficando em 3,7%.
- As medições foram automatizadas pelo script `rodar_experimentos.sh` e as estatísticas e gráficos gerados pelo `analise.py`.

Tempos de execução

Todas as configurações ficaram com coeficiente de variação abaixo de 5%, dentro do limite aceito.

Plataforma 1 — Codespaces (mono-processada)

Threads	Exec 1 (s)	Exec 2 (s)	Exec 3 (s)	Exec 4 (s)	Exec 5 (s)	Média (s)	Desvio (s)	CV (%)
1	1,451	1,455	1,467	1,440	1,437	1,450	0,012	0,83
2	1,452	1,419	1,468	1,464	1,467	1,454	0,021	1,42
4	1,435	1,437	1,436	1,419	1,449	1,435	0,011	0,75
8	1,425	1,508	1,447	1,426	1,417	1,445	0,037	2,57
16	1,441	1,458	1,426	1,449	1,471	1,449	0,017	1,17
32	1,421	1,427	1,418	1,450	1,425	1,428	0,013	0,89

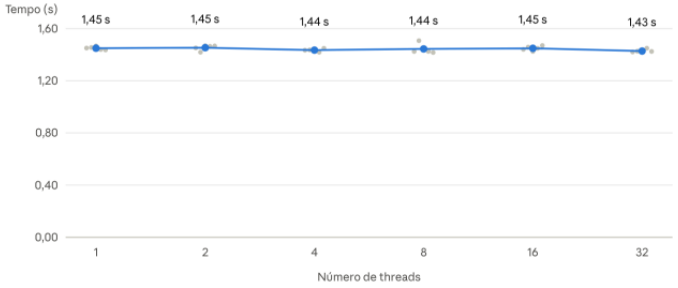
Plataforma 2 — Xeon 2 núcleos (dual-processada)

Threads	Exec 1 (s)	Exec 2 (s)	Exec 3 (s)	Exec 4 (s)	Exec 5 (s)	Média (s)	Desvio (s)	CV (%)
1	1,281	1,265	1,392	1,264	1,247	1,290	0,058	4,53
2	0,666	0,650	0,660	0,624	0,614	0,643	0,023	3,54
4	0,664	0,649	0,713	0,665	0,689	0,676	0,025	3,72
8	0,635	0,642	0,646	0,653	0,697	0,655	0,025	3,76
16	0,644	0,653	0,694	0,645	0,646	0,656	0,021	3,25
32	0,626	0,627	0,639	0,669	0,624	0,637	0,019	2,96

Tempo médio por plataforma

Plataforma 1 (mono): o tempo fica em 1,45s com qualquer número de threads

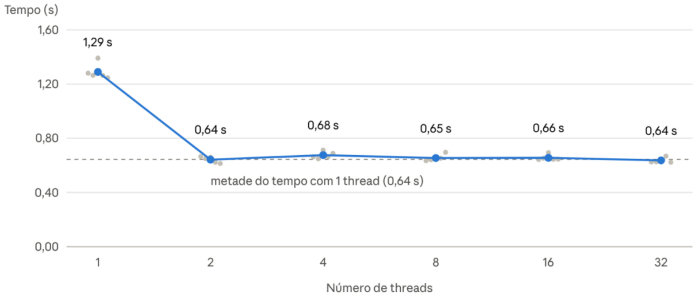
> Linha: tempo médio de 5 execuções, pontos cinza: cada execução, 10^9 termos



Na plataforma 1 o tempo médio ficou entre 1,428 e 1,454 s em todas as configurações, uma variação menor que 2%, do tamanho da própria dispersão das medidas. Com um único núcleo físico, as threads apenas se revezam nele: o trabalho total é o mesmo e não há ganho. Também não há perda visível, porque criar até 32 threads e alternar entre elas custa muito pouco perto de 1,4 s de cálculo. As 2 CPUs lógicas do Codespaces (hyperthreading) não ajudaram: o laço é limitado pela divisão em ponto flutuante, e as duas threads de hardware disputam a mesma unidade de cálculo do núcleo.

Plataforma 2 (dual): 2 threads cortam o tempo pela metade, mais threads não ajudam

> Linha: tempo médio de 5 execuções, pontos cinza: cada execução, 10^9 termos

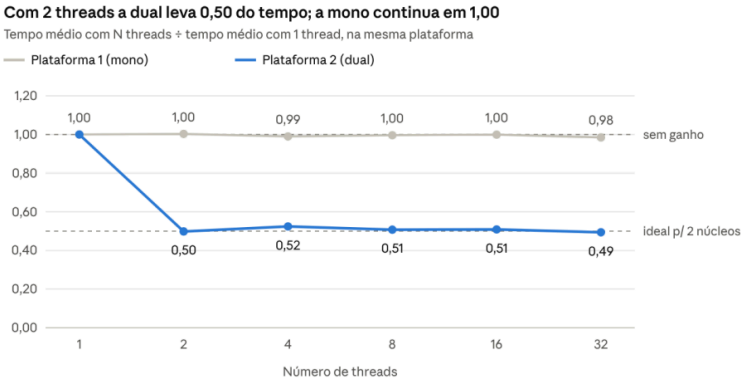


Na plataforma 2, o tempo caiu de 1,290 s com 1 thread para 0,643 s com 2, um speedup de 2,01, igual ao número de núcleos. A partir de 4 threads o tempo fica estável entre 0,637 e 0,676 s (speedup entre 1,91 e 2,03): não há mais núcleos livres, e as threads extras só dividem os mesmos dois núcleos. O tempo com 1 thread também é menor que na plataforma 1 (1,290 s contra 1,450 s), sinal de um núcleo mais rápido ou não compartilhado.

Comparação normalizada

Com 2 threads a dual leva 0,5 do tempo e a mono continua em 1,0

> Tempo médio com N threads / tempo médio com 1 thread, na mesma plataforma



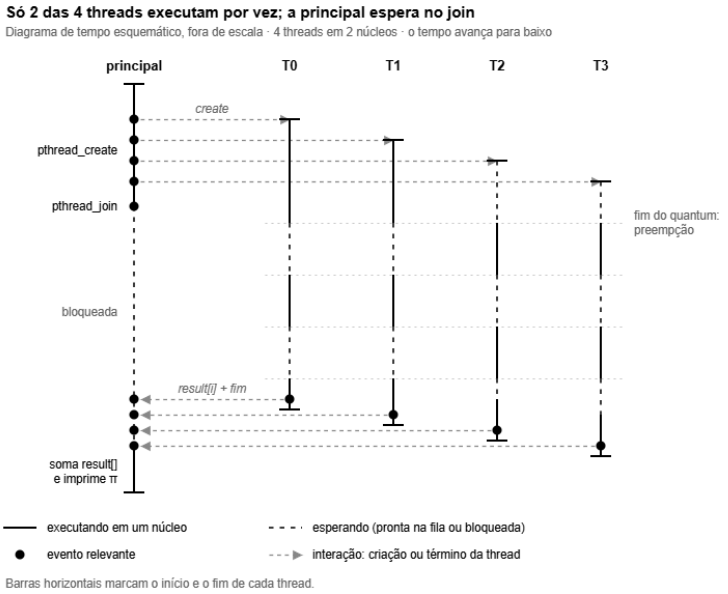
Normalizar cada curva pelo seu tempo com 1 thread tira da comparação a diferença de velocidade dos processadores e deixa só o efeito do paralelismo. A curva da plataforma 2 encosta na linha ideal de 0,50 com 2 threads e fica nela até 32 threads: o ganho máximo é limitado pelo número de núcleos físicos, não pelo de threads. A curva da plataforma 1 fica sobre a linha de 1,00 do início ao fim.

Três pontos merecem crítica:

- **Ganho perto do ideal.** O problema é de paralelismo perfeito: as somas parciais são independentes, não há comunicação nem sincronização entre as threads, e a parte sequencial (criar threads e somar o vetor) é desprezível. Por isso o speedup chega a 2,0 em 2 núcleos, sem a perda que a Lei de Amdahl prevê para programas com trechos sequenciais maiores.
- **Excesso de threads não custa caro.** Com 32 threads em 2 núcleos, o tempo normalizado ficou em 0,49, igual ao de 2 threads. O custo de criar as threads e das trocas de contexto não aparece na escala de 1,3 s. Ele só apareceria com muito mais threads ou com menos trabalho por thread.
- **CPUs lógicas não são núcleos.** As duas plataformas têm 2 CPUs lógicas, mas só a plataforma 2 ganhou com o paralelismo. Para um laço limitado por cálculo em ponto flutuante, o que conta é o número de núcleos físicos; o `nproc` sozinho não basta para prever o desempenho.

Uma limitação do experimento: as duas plataformas são máquinas virtuais em nuvem, onde outros clientes podem disputar o mesmo hardware. Os coeficientes de variação abaixo de 5% indicam que essa interferência não distorceu as medidas.

Diagrama de tempo



A thread principal cria as threads de cálculo e fica bloqueada em `pthread_join` até a última terminar; só então soma o vetor `result[]`. Como há 4 threads e 2 núcleos, o escalonador mantém 2 em execução e troca o par a cada fim de quantum. Por isso o tempo total com 4 threads é o mesmo que com 2: o trabalho por núcleo não muda.

Questões

1. As threads implementadas são preemptivas ou cooperativas?

São **preemptivas**. O código das threads é só um laço de cálculo: nenhuma delas chama `sched_yield`, dorme ou faz E/S, ou seja, nenhuma cede a CPU voluntariamente. Mesmo assim, com 32 threads em 1 ou 2 núcleos, todas progridem e terminam, e o tempo total não cresce. Isso só acontece porque o escalonador do sistema operacional interrompe cada thread ao fim do seu quantum e passa a CPU para outra. Se as threads fossem cooperativas, a primeira a ganhar a CPU executaria seu bloco inteiro antes de qualquer outra começar.

2. Que modelo de threads o sistema operacional implementa (N:1, 1:1 ou N:M)? Como isso pode ser deduzido dos experimentos?

O Linux implementa o modelo **1:1** (biblioteca NPTL): cada thread criada com `pthread_create` corresponde a uma thread do kernel, escalonada de forma independente.

Os experimentos mostram isso pela plataforma 2. Com 2 threads, o tempo caiu de 1,290 s para 0,643 s, um speedup de 2,0, igual ao número de núcleos. Para isso acontecer, as duas threads precisam estar executando ao mesmo tempo em núcleos diferentes, e só o kernel pode colocá-las lá. No modelo **N:1**, todas as threads do processo seriam uma única entidade para o kernel, ocupariam um só núcleo, e o tempo nunca cairia. A curva da plataforma 2 seria igual à da plataforma 1.

O comportamento acima de 2 threads também é coerente com 1:1. O tempo fica estável porque não há mais núcleos livres, e o custo de criar e alternar até 32 threads do kernel é desprezível perto de 1,3 s de cálculo. O modelo N:M também permitiria paralelismo, mas não é usado no Linux atual; a distinção entre 1:1 e N:M não aparece nos tempos medidos e vem da documentação da NPTL.